

全部完成 ✅ 项目已生成在 C:\Users\ywyuan\Desktop\anker-asr-wenet\, 所有代码文件都已写好并通过语法检查, 教学版 U2++ 模型还实跑验证了一遍 (前向 + 三路损失都正常)。

已生成的文件清单

```
anker-asr-wenet/
├── README.md          # ★ 总教程：环境、一键运行、分步详解、FAQ
├── requirements.txt    # Python 依赖
├── conf/
│   └── train_u2pp_conformer.yaml # ★ U2++ Conformer 训练配置 (conformer+bitransformer+ctc)
├── local/
│   ├── generate_anker_corpus.py # ★ 用 TTS 自动合成「安克指令」语音数据集
│   └── prepare_data.py         # 生成 data.list + 汉字字典
├── tools/
│   └── make_cmvn.py           # 计算 CMVN 特征归一化
├── model/
│   └── u2pp_conformer_min.py  # ★ 从零实现的精简 U2++ (已实跑通过, 讲原理用)
├── run.ps1             # ★ Windows 一键脚本 (你用这个)
├── run.sh              # Linux/Mac/WSL 一键脚本
├── recognize.py        # 测试集解码 + 自动算 CER (字错误率)
├── infer_demo.py      # ★ 单条音频 / 麦克风实时识别 demo
└── docs/
    ├── 原理讲解.md     # Conformer/CTC/U2++ 原理详解 (报告素材)
    └── 答辩大纲.md     # PPT 结构 + 答辩高频问答 + 参考文献
```

你接下来只需 3 步

1. 装环境 (README 第 2 节) : ffmpeg + pip install -r requirements.txt + clone WeNet 仓库 pip install -e .
2. 一键跑: cd 桌面\anker-asr-wenet 然后 .\run.ps1 —— 会自动 合成数据 → 准备数据 → 训练 U2++ → 解码算 CER → 演示识别
3. 写报告: 直接用 docs/原理讲解.md 和 docs/答辩大纲.md

几个我替你做的关键决策 (答辩可能问到, 先告诉你)

- 数据来源: 小白没有数据集, 所以用 edge-tts 多发音人合成 + WeNet 速度扰动/SpecAugment 数据增强, 全程不用录音、不用下载大数据集就能跑通。这一点在 docs/答辩大纲.md 的 Q4 已经帮你准备好怎么解释。
- 模型确实是 U2++: Conformer 编码器 (因果卷积+动态chunk, 支持流式) + CTC 分支 + 双向 bitransformer 解码器 (L2R + R2L) + attention_rescoring 重打分, 全是你最新的工程技术。
- 参数调小了 (output_size 144、6 层) 让 CPU 也能训完; 想要效果更好, README 里写了怎么往大调。

⚠️ 唯一需要你留意的: WeNet 不同版本的训练参数偶有差异, 如果 python -m wenet.bin.train 报某个字段错, 按报错删/改即可, README 的 FAQ 和原理讲解里都有说明。需要我帮你适配某个具体的 WeNet 版本, 或者把指令清单换成你想要的内容, 告诉我就行。

完整教程已写到 ?? [anker-asr-wenet/docs/autodl手搓教程.md](#) (你可以直接拖到 JupyterLab 里对照看, 或者照下面的内容跟着做)

？ AutoDL 上最容易踩的 5 个坑 (先记住这几条, 能省你几个小时)

1. 项目和数据一定放 `/root/autodl-tmp/` —— 这是数据盘, 关机不丢、空间大。放 `/root/` (系统盘) 的东西, 重置镜像会被清空, 而且容易爆盘。
2. 访问 [GitHub](#) 和微软 TTS 必须开学术加速:
`source /etc/network_turbo`
2. 不开的话 `git clone wenet` 和 `edge-tts` 生成数据都会超时失败。
3. 务必确认 PyTorch 是 GPU 版。装完跑这条, 必须看到 `True`:
`python -c "import torch; print('CUDA可用:', torch.cuda.is_available())"`
3. 是 `False` 就是装成 CPU 版了, 要用 `--index-url https://download.pytorch.org/whl/cu121` 重装。
4. 服务器没有麦克风 —— `infer_demo.py --mic` 用不了, 只能用 `--wav` 文件路径。想识别自己的录音, 在电脑录好 `wav` 拖上传即可
5. 训练完立刻关机省钱, 模型/数据存在数据盘。

这份 Linux 教程和 Windows 版的关键差异

方面	Windows	AutoDL (Linux)
命令换行符	<code>`</code>	<code>\</code>
路径分隔	<code>\</code>	<code>/</code>
装 ffmpeg	<code>winget install</code>	<code>apt-get install -y ffmpeg</code>
激活环境	<code>.venv\Scripts\Activate.ps1</code>	<code>conda activate</code> 或 <code>source ../bin/activate</code>
训练	CPU, 慢	GPU, <code>export CUDA_VISIBLE_DEVICES=0</code> , 快很多
编辑文件	本地 IDE	JupyterLab / VSCode Remote / nano

训练这一步要特别注意 (第 9 部分)

不同 WeNet 版本的训练启动方式不一样, 我在教程里给了两个方案, 先试 A 不行用 B:

- 方案 A: `python -m wenet.bin.train ...` (多数版本可用)
- 方案 B: `torchrun --standalone --nproc_per_node=1 ../train.py --train_engine torch_ddp ...` (较新版本要求分布式启动)

如果报 `unrecognized arguments`, 先跑 `python -m wenet.bin.train --help` 看你这版支持哪些参数, 或参考你 clone 下来的 `wenet/examples/aishell/s0/run.sh` ——那是和你版本完全匹配的官方范例。

另外想后台挂着训练、断开 SSH 也不停, 用:

```
nohup python -m wenet.bin.train ... (参数) ... > train.log 2>&1 &
tail -f train.log
```

你现在按教程走到哪一步了? 如果某一步报错, 把完整报错信息贴给我 (尤其是训练那步, 版本差异最大), 我帮你对症改命令或改配置。

明白！你想自己一行行把这个项目"手搓"出来，边搓边理解。我给你写一份**超详细的手把手教程**——它的核心思路是：**分阶段建文件 + 每建一个就立刻运行验证**，这样你不会"全写完才发现某处错了"。

桌面上我之前生成的文件就当作"标准答案/对照参考"，教程里**不再重复贴没改过的代码**，只告诉你：这个文件干嘛的、放哪、怎么跑、跑出来该看到什么、错了怎么办。

我把它写成一个文档放进项目里，方便你边看边做：

```
● Write(docs\手搓教程.md)
? Wrote 465 lines to docs\手搓教程.md
1 # ?? 手搓教程：从零搭建「安克耳机语音识别 (WeNet U2++)」
2
3 > 适合人群：**完全小白**。你只要会用鼠标、会打字、会复制粘贴就能跟下来。
4 > 工具：VSCode 或 PyCharm 任选其一（本教程两个都给步骤）。系统：Windows。
5 >
6 > **核心理念：分阶段做，每做完一步立刻运行验证（看到 ? 再进行下一步）。**
7 > 千万不要一口气把所有文件都建好才运行——那样出错很难定位。
8 >
9 > 桌面上 `anker-asr-wenet/` 里我已经放了全部"标准答案"文件。本教程带你
10 > **理解并亲手重建**它们。每个文件你可以：
... +455 lines (ctrl+o to expand)
```

已经帮你写好了 ?? [anker-asr-wenet/docs/手搓教程.md](#)

这份教程是专门为"自己一行行搓出来"设计的，和之前的 README 不一样的地方在于：

它的核心特点

1. **分阶段建文件 + 每步立刻验证**：不是"全写完再跑"，而是**建一个文件 → 马上运行 → 看到 ? 再继续**。每一步我都写了"检查点"，告诉你该看到什么输出、文件夹里该多出什么文件。这样出错能立刻定位在哪一步。
2. **VSCode 和 PyCharm 两种都给了步骤**：建项目、建虚拟环境、选解释器、用按钮运行——两个 IDE 的菜单点法都写了。
3. **建文件的顺序是精心排过的**（让你尽早有成就感）：
 - 先跑**教学版 U2++ 模型**（第 3 部分）——不需要装 WeNet、不需要数据，立刻就能"看见"Conformer + CTC + 双向解码器跑起来，建立信心。
 - 再一步步：生成数据 → 准备格式 → CMVN → 装 WeNet → 训练 → 解码 → 麦克风 demo。
4. **没改的代码不重复贴**：教程里每个文件只说明"它干嘛的、你要看懂哪几个关键点、怎么跑、错了怎么办"，代码内容让你对照桌面上已有的标准答案文件。
5. **附了两张速查表**：结尾的"全流程回顾表"（建什么文件→跑什么命令→看到什么）和"报错急救表"（9 种常见报错的原因和解决）。

建议你这样用

- 打开这个 [.md](#) 文件（VSCode 里按 **Ctrl+Shift+V** 可以预览渲染效果，PyCharm 右上角也有预览按钮），左边看教程、右边敲代码。
- 严格按"检查点 ?"推进，**没看到 ? 就别往下走**，先对照急救表排查。
- 第 8 部分训练那里要特别注意：不同 WeNet 版本配置字段偶有差异，如果报 **unexpected key**，按报错删/改 yaml 对应行，核心结构别动。

需要我把教程里某一部分讲得更细（比如虚拟环境、或者训练参数怎么调），或者你跑到哪一步卡住了，直接把报错贴给我，我帮你